

BIPES DSL Python Source Files

Only the active Python library files listed in server/block_dsl/integrate.py.

[static/page/blocks/libraries/PicoRobotics.py](#)

```
import machine
import utime

# @block {
#   "category": "Robotics Board DSL",
#   "color": 45,
#   "url": "https://github.com/KitronikLtd/Kitronik-Pico-Robotics-Board-MicroPython",
#   "instanceMode": "singleton",
#   "instanceName": "board",
#   "methodInstanceMode": "fixed_name"
# }
# Pico Robotics Board blocks generated from the Python library.
class KitronikPicoRobotics:
    #Class variables - these should be the same for all instances of the class.
    # If you wanted to write some code that stepped through
    # the servos or motors then this is the Base and size to do that
    SRV_REG_BASE = 0x08
    MOT_REG_BASE = 0x28
    REG_OFFSET = 4
    PRESCALE_VAL = b'\x79'
    PI_ESTIMATE = 3.1416

    # Set up the PCA chip for 50Hz servo timing and reset its output registers.
    def initPCA(self):
        # Make sure we are in a known position
        # Soft reset of the I2C chip
        self.i2c.writeto(0, "\x06")

        # setup the prescale to have 20mS pulse repetition - this is dictated by the servos.
        # set PWM Frequency Pre Scale. The prescale value is determined with the formula:
        # prescale value = round(osc clock / (4096 * update rate))
        # Where update rate is the output modulation frequency required.
        # For example, the output frequency of 50Hz (20ms) for the servo, with the internal oscillator
        # clock frequency of 25 Mhz is as follows:
        # prescale value = round( 25MHZ / (4096 * 50Hz) ) - 1
        # prescale value = round( 25000000 / (4096 * 50) ) - 1
        # prescale value = 121 = 79h = 0x79
        self.i2c.writeto_mem(108, 0xfe, self.PRESCALE_VAL)

        #block write outputs to off
        self.i2c.writeto_mem(108, 0xfa, "\x00")
        self.i2c.writeto_mem(108, 0xfb, "\x00")
        self.i2c.writeto_mem(108, 0xfc, "\x00")
        self.i2c.writeto_mem(108, 0xfd, "\x00")

        # come out of sleep
        self.i2c.writeto_mem(108, 0x00, "\x01")

        # It takes 500uS max for the oscillator to be up and running once the SLEEP bit (bit 4) has
        # been set to logic 0. Timings on outputs are not guaranteed if the PWM control registers are
        # accessed within the 500uS window.
        utime.sleep_us(500)

    # Adjusts the servos.
    # This block should be used if the connected servo does not respond correctly to the 'servoWrite' command.
    # Try changing the value by small amounts and testing the servo until it correctly sets to the angle.
    # @block {
    #   "params": {
    #     "change": {"checkType": "Number"}
    #   }
    # }
    # Adjust the global servo calibration by a small amount.
```

```

def adjustServos(self, change):
    if change < -25:
        change = -25
    if change > 25:
        change = 25
    self.PRESCALE_VAL = (121 + change).to_bytes(1,"big")
    self.initPCA()

# To get the PWM pulses to the correct size and zero
# offset these are the default numbers.
#Servo multiplier is calculated as follows:
# 4096 pulses ->20ms 1ms-> count of 204.8
# 1ms is 90 degrees of travel, so each degree is a count of 204.8/90->2.2755
# servo pulses always have a minimum value - so there is guarantee to be a pulse.
# in the servos I've examined this is 0.5ms or a count of 102
# to calculate the count for the correct pulse is simply:
# (degrees x count per degree) + offset
# @block {
#     "label": "turn servo {servo} to {degrees} degrees",
#     "params": {
#         "servo": {"checkType": "Number"},
#         "degrees": {"checkType": "Number"}
#     }
# }

# Turn a servo motor to the requested angle in degrees.
def servoWrite(self, servo, degrees):
    #check the degrees is a reasonable number. we expect 0-180, so cap at those values.
    if(degrees>180):
        degrees = 180
    elif (degrees<0):
        degrees = 0
    #check the servo number
    if((servo<1) or (servo>8)):
        raise Exception("INVALID SERVO NUMBER") #harsh, but at least you'll know
    calcServo = self.SRV_REG_BASE + ((servo - 1) * self.REG_OFFSET)
    PWMVal = int((degrees*2.2755)+102) # see comment above for maths
    lowByte = PWMVal & 0xFF
    highByte = (PWMVal>>8)&0x01 #cap high byte at 1 - should never be more than 2.5ms.
    self.i2c.writeto_mem(self.CHIP_ADDRESS, calcServo,bytes([lowByte]))
    self.i2c.writeto_mem(self.CHIP_ADDRESS, calcServo+1,bytes([highByte]))

# Takes the servo to change and the angle in radians to move to.
# 0 radians to 3.1416
# @block {
#     "params": {
#         "servo": {"checkType": "Number"},
#         "radians": {"checkType": "Number"}
#     }
# }

# Turn a servo motor to the requested angle in radians.
def servoWriteRadians(self, servo, radians):
    if servo < 1:
        servo = 1
    if servo > 8:
        servo = 8
    if radians < 0:
        radians = 0
    if radians > self.PI_ESTIMATE:
        radians = self.PI_ESTIMATE

    calcServo = self.SRV_REG_BASE + ((servo - 1) * self.REG_OFFSET)
    PWMVal = int((radians / self.PI_ESTIMATE) * 408) + 102 # See comment above for maths
    lowByte = PWMVal & 0xFF
    highByte = (PWMVal >> 8) & 0x01 # Cap high byte at 1 - should never be more than 2.5ms
    self.i2c.writeto_mem(self.CHIP_ADDRESS, calcServo, bytes([lowByte]))
    self.i2c.writeto_mem(self.CHIP_ADDRESS, calcServo + 1, bytes([highByte]))

#Driving the motor is simpler than the servo - just convert 0-100% to 0-4095 and push it to the correct registers.
#each motor has 4 writes - low and high bytes for a pair of registers.
# @block {
#     "label": "start dc motor {motor} direction {direction} speed {speed}",
#     "params": {

```

```

#     "motor": {"checkType": "Number"},
#     "direction": {
#         "inputKind": "field_dropdown",
#         "options": [ ["Forward", "f"], ["Reverse", "r"] ]
#     },
#     "speed": {"checkType": "Number"}
# }
# }
# Start a DC motor with direction and speed control.
def motorOn(self, motor, direction, speed):
    # cap speed to 0-100%
    if (speed < 0):
        speed = 0
    elif (speed > 100):
        speed = 100

    if ((motor < 1) or (motor > 4)):
        raise Exception("INVALID MOTOR NUMBER") # harsh, but at least you'll know

    motorReg = self.MOT_REG_BASE + (2 * (motor - 1) * self.REG_OFFSET)
    PWMVal = int(speed * 40.95)
    lowByte = PWMVal & 0xFF
    highByte = (PWMVal >> 8) & 0xFF # motors can use all 0-4096
    # print (motor, direction, "LB ", lowByte, " HB ", highByte)
    if direction == "f":
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg, bytes([lowByte]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+1, bytes([highByte]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+4, bytes([0]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+5, bytes([0]))
    elif direction == "r":
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+4, bytes([lowByte]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+5, bytes([highByte]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg, bytes([0]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+1, bytes([0]))
    else:
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+4, bytes([0]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+5, bytes([0]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg, bytes([0]))
        self.i2c.writeto_mem(self.CHIP_ADDRESS, motorReg+1, bytes([0]))
        raise Exception("INVALID DIRECTION")

    # To turn off set the speed to 0...
    # @block {
    #     "label": "stop dc motor {motor}",
    #     "params": {
    #         "motor": {"checkType": "Number"}
    #     }
    # }
    # Stop a DC motor.
    def motorOff(self, motor):
        self.motorOn(motor, "f", 0)

#####
# Stepper Motors
#####
# this is only a basic full stepping.
# speed sets the length of the pulses (and hence the speed...)
# so is 'backwards' - the fastest that works reliably with the motors I have to hand is 20mS, but slower than that is
good. tested to 2000 (2 seconds per step).
# motor should be 1 or 2 - 1 is terminals for motor 1 and 2 on PCB, 2 is terminals for motor 3 and 4 on PCB
# @block {
#     "params": {
#         "motor": {"checkType": "Number"},
#         "direction": {
#             "inputKind": "field_dropdown",
#             "options": [ ["Forward", "f"], ["Reverse", "r"] ]
#         },
#         "steps": {"checkType": "Number"},
#         "speed": {"checkType": "Number", "defaultValue": 20},
#         "holdPosition": {"checkType": "Boolean", "defaultValue": false}
#     }
# }
# }
# Step a motor a fixed number of full steps.

```

```

def step(self,motor, direction, steps, speed =20, holdPosition=False):

    if((motor<1) or (motor>2)):
        raise Exception("INVALID MOTOR NUMBER") # harsh, but at least you'll know

    if(direction == "f"):
        directions = ["f", "r"]
        coils = [((motor*2)-1),(motor*2)]
    elif (direction == "r"):
        directions = ["r", "f"]
        coils = [(motor*2),((motor*2)-1)]
    else:
        raise Exception("INVALID DIRECTION") #harsh, but at least you'll know
    while steps > 0:
        for direction in directions:
            if(steps == 0):
                break
            for coil in coils:
                self.motorOn(coil,direction,100)
                utime.sleep_ms(speed)
                steps -=1
                if(steps == 0):
                    break
#to save power turn off the coils once we have finished.
#this means the motor wont hold position.
    if(holdPosition == False):
        for coil in coils:
            self.motorOff(coil)

#Step an angle. this is limited by the step resolution - so 200 steps is 1.8 degrees per step for instance.
# a request for 20 degrees with 200 steps/rev will result in 11 steps - or 19.8 rather than 20.
# @block {
#     "params": {
#         "motor": {"checkType": "Number"},
#         "direction": {
#             "inputKind": "field_dropdown",
#             "options": [ ["Forward", "f"], ["Reverse", "r"] ]
#         },
#         "angle": {"checkType": "Number"},
#         "speed": {"checkType": "Number", "defaultValue": 20},
#         "holdPosition": {"checkType": "Boolean", "defaultValue": false},
#         "stepsPerRev": {"checkType": "Number", "defaultValue": 200}
#     }
# }
# Step a motor by a requested angle.
def stepAngle(self,motor, direction, angle, speed =20, holdPosition=False, stepsPerRev=200):
    steps = int(angle/(360/stepsPerRev))
    print (steps)
    self.step(motor, direction, steps, speed, holdPosition)

# initialaisation code for using:
# defaults to the standard pins and address for the kitronik board, but could be overridden
# @block {
#     "label": "create robotics board address {I2CAddress} sda {sda} scl {scl}",
#     "params": {
#         "I2CAddress": {"checkType": "Number"},
#         "sda": {"checkType": "Number"},
#         "scl": {"checkType": "Number"}
#     }
# }
# Create and initialize the Kitronik Pico Robotics board.
def __init__(self, I2CAddress=108,sda=8,scl=9):
    self.CHIP_ADDRESS = 108
    sda=machine.Pin(sda)
    scl=machine.Pin(scl)
    self.i2c=machine.I2C(0,sda=sda, scl=scl, freq=100000)
    self.initPCA()

```

```

import time
import math
from machine import Pin
import stepper

# @block {
#   "category": "Sand Drawing Machine DSL",
#   "color": 20,
#   "instanceMode": "multiple",
#   "methodInstanceMode": "key_input"
# }
# Sand table robot blocks generated from the Python library.
class SandTableRobot:
    """
    Encapsulates all robot control.

    Public API:
    - home()
    - off()
    - move_line()
    - move_arc()
    - draw_spiral()
    - draw_flower()
    - run_gcode_file(path, ...)
    - run_gcode_text(gcode_text, ...)
    """

    # -----
    # Internal Stepper class (your original)
    # -----
    class Stepper:
        def __init__(self, in1, in2, in3, in4, backlash_steps=0):
            self.pins = [Pin(in1, Pin.OUT), Pin(in2, Pin.OUT), Pin(in3, Pin.OUT), Pin(in4, Pin.OUT)]
            self.step_sequence = [
                [1, 0, 0, 0], [1, 1, 0, 0], [0, 1, 0, 0], [0, 1, 1, 0],
                [0, 0, 1, 0], [0, 0, 1, 1], [0, 0, 0, 1], [1, 0, 0, 1]
            ]
            self.step_count = len(self.step_sequence)
            self.current_step = 0
            self.backlash_steps = backlash_steps
            self.last_direction = 0
            self.off()

        def _set_step(self, step_data):
            for i in range(4):
                self.pins[i].value(step_data[i])

        def off(self):
            for pin in self.pins:
                pin.value(0)

        def step(self, direction):
            # IMPORTANT: this Stepper.step() takes +/-1 direction, not N steps.
            # We will provide a helper to step N times.
            if self.last_direction != 0 and direction != self.last_direction:
                for _ in range(self.backlash_steps):
                    self.current_step = (self.current_step + direction) % self.step_count
                    self._set_step(self.step_sequence[self.current_step])
                    time.sleep_ms(2)

            self.last_direction = direction
            self.current_step = (self.current_step + direction) % self.step_count
            self._set_step(self.step_sequence[self.current_step])

    # -----
    # Constructor / configuration
    # -----
    # @block {
    #   "label": "create sand robot # {id} shoulder motor pins {motor1_pins} elbow motor pins {motor2_pins} shoulder

```

```

sensor {sensor_shoulder_pin} elbow sensor {sensor_elbow_pin}",
#   "params": {
#       "id": {"checkType": "Number", "defaultValue": 1},
#       "motor1_pins": {"defaultValue": [17, 16, 15, 14]},
#       "motor2_pins": {"defaultValue": [21, 20, 19, 18]},
#       "sensor_shoulder_pin": {"checkType": "Number"},
#       "sensor_elbow_pin": {"checkType": "Number"},
#       "L1": {"checkType": "Number"},
#       "L2": {"checkType": "Number"},
#       "steps_per_rev": {"checkType": "Number"},
#       "backlash_deg_m1": {"checkType": "Number"},
#       "backlash_deg_m2": {"checkType": "Number"},
#       "homing_dir_shoulder": {"checkType": "Number"},
#       "homing_dir_elbow": {"checkType": "Number"},
#       "homing_clear_steps": {"checkType": "Number"},
#       "default_speed_ms": {"checkType": "Number"}
#   }
# }
# Create a sand table robot instance.
def __init__(
    self,
    motor1_pins, # shoulder motor pins
    motor2_pins, # elbow motor pins
    sensor_shoulder_pin,
    sensor_elbow_pin,
    id=1,
    L1=31.0,
    L2=31.0,
    steps_per_rev=4096.0,
    backlash_deg_m1=3.2,
    backlash_deg_m2=3.0,
    homing_dir_shoulder=1,
    homing_dir_elbow=1,
    homing_clear_steps=500,
    default_speed_ms=2
):
    self.L1 = float(L1)
    self.L2 = float(L2)
    self.STEPS_PER_REV = float(steps_per_rev)
    self.id = id

    self.HOMING_DIR_SKULDER = int(homing_dir_shoulder)
    self.HOMING_DIR_ALBUE = int(homing_dir_elbow)
    self.HOMING_CLEAR_STEPS = int(homing_clear_steps)
    self.default_speed_ms = int(default_speed_ms)

    self.BACKLASH_STEPS_1 = int((float(backlash_deg_m1) / 360.0) * self.STEPS_PER_REV)
    self.BACKLASH_STEPS_2 = int((float(backlash_deg_m2) / 360.0) * self.STEPS_PER_REV)

    # Cartesian state (global XY)
    self.last_x = self.L1 + self.L2
    self.last_y = 0.0

    # "IK step target" state (your lastShoulderSteps/lastElbowSteps)
    self.lastShoulderSteps = 0
    self.lastElbowSteps = 0

    self.prev_theta1 = 0
    self.prev_theta2 = 0

    self.sensor_skulder = Pin(int(sensor_shoulder_pin), Pin.IN, Pin.PULL_UP)
    self.sensor_albue = Pin(int(sensor_elbow_pin), Pin.IN, Pin.PULL_UP)

    m1 = motor1_pins
    m2 = motor2_pins
    self.skulder = stepper.HalfStepMotor.frompins(m1[0], m1[1], m1[2], m1[3])
    self.albue = stepper.HalfStepMotor.frompins(m2[0], m2[1], m2[2], m2[3])

    # G-code modal state
    self._g_abs = True
    self._g_units_mm = True
    self._g_last_motion = None # 0 or 1

```

```

# -----
# Internal helpers
# -----
# @block {
#   "label": "sand robot # {id} turn motors off"
# }
# Turn off both motors for a selected robot instance.
def off(self):
    self.skulder.reset()
    self.albue.reset()

def _rad_to_steps(self, radians):
    # Keep it identical to your code (int trunc). You can switch to round() later if desired.
    return int((radians / (2 * math.pi)) * self.STEPS_PER_REV)

def _step_n(self, stepper_obj, steps, speed_ms):
    """
    Since Stepper.step() is +/-1 only, expand N steps here.
    Positive steps => direction +1 repeated.
    Negative steps => direction -1 repeated.
    """
    if steps == 0:
        return
    direction = 1 if steps > 0 else -1
    for _ in range(abs(steps)):
        stepper_obj.step(direction)
        if speed_ms > 0:
            time.sleep_ms(speed_ms)

def _unwrap(self, a, prev):
    # shift a by  $\pm 2\pi$  until it is closest to prev
    while a - prev > math.pi:
        a -= 2 * math.pi
    while a - prev < -math.pi:
        a += 2 * math.pi
    return a

def _InverseKinematics(self, x, y):
    r2 = x*x + y*y
    r = math.sqrt(r2)

    # Reachability (match your logic)
    if r > (self.L1 + self.L2) or r < 1e-6:
        return None

    c2 = (r2 - self.L1*self.L1 - self.L2*self.L2) / (2 * self.L1 * self.L2)
    c2 = max(-1.0, min(1.0, c2))
    theta2 = math.acos(c2)

    k1 = self.L1 + self.L2 * math.cos(theta2)
    k2 = self.L2 * math.sin(theta2)
    theta1 = math.atan2(y, x) - math.atan2(k2, k1)
    #print(theta1, theta2)

    return (theta1, theta2)

def _moveSteppersCount(self, shoulder, elbow):
    shoulderSteps = self._rad_to_steps(shoulder)
    elbowSteps = self._rad_to_steps(elbow)
    return shoulderSteps, elbowSteps

# -----
# Internal G-code parsing helpers (unchanged)
# -----
def _strip_comment(self, line):
    i = line.find(';')
    if i != -1:
        line = line[:i]
    return line.strip()

def _parse_tokens(self, line):

```

```

out = {}
i = 0
n = len(line)

while i < n:
    c = line[i]

    if c in (' ', '\t'):
        i += 1
        continue

    if ('A' <= c <= 'Z') or ('a' <= c <= 'z'):
        letter = c.upper()
        i += 1

    while i < n and line[i] in (' ', '\t'):
        i += 1

    j = i
    if j < n and line[j] in ('+', '-'):
        j += 1

    dot = False
    while j < n:
        ch = line[j]
        if '0' <= ch <= '9':
            j += 1
        elif ch == '.' and not dot:
            dot = True
            j += 1
        else:
            break

    if j > i:
        try:
            out[letter] = float(line[i:j])
        except:
            pass

    i = j
    else:
        i += 1

return out

def _gcode_reset_modal(self):
    self._g_abs = True
    self._g_units_mm = True
    self._g_last_motion = None

def _gcode_process_line(self, raw_line, segments=40, draw_speed_ms=None, travel_speed_ms=None):
    if draw_speed_ms is None:
        draw_speed_ms = self.default_speed_ms
    if travel_speed_ms is None:
        travel_speed_ms = self.default_speed_ms

    line = self._strip_comment(raw_line)
    if not line:
        return False

    t = self._parse_tokens(line)

    if 'G' in t:
        g = int(t['G'])
        if g == 90:
            self._g_abs = True
            return False
        if g == 91:
            self._g_abs = False
            return False
        if g == 21:
            self._g_units_mm = True

```



```

        return False
    if g == 20:
        self._g_units_mm = False
        return False
    if g in (0, 1):
        self._g_last_motion = g

    if self._g_last_motion not in (0, 1):
        return False

    if ('X' not in t) and ('Y' not in t):
        return False

    rapid = (self._g_last_motion == 0)
    speed = travel_speed_ms if rapid else draw_speed_ms

    def to_mm(v):
        return v if self._g_units_mm else (v * 25.4)

    tx = self.last_x
    ty = self.last_y

    if 'X' in t:
        vx = to_mm(t['X'])
        tx = vx if self._g_abs else (self.last_x + vx)

    if 'Y' in t:
        vy = to_mm(t['Y'])
        ty = vy if self._g_abs else (self.last_y + vy)

    self.move_line(tx, ty, segments=segments, speed=speed)
    return True

# -----
# Public: Homing
# -----
# @block {
#   "label": "sand robot # {id} home"
# }
# Home the selected sand robot.
def home(self):
    print("Starter Homing...")

    print("Homing Skulder (M1)...")
    for _ in range(self.HOMING_CLEAR_STEPS):
        self.skulder.step(-self.HOMING_DIR_SKULDER)
        time.sleep_ms(2)

    while self.sensor_skulder.value() == 1:
        self.skulder.step(self.HOMING_DIR_SKULDER)
        time.sleep_ms(3)

    self.skulder.reset()
    print(">> Skulder Nulstillet!")

    print("Homing Albue (M2)...")
    for _ in range(self.HOMING_CLEAR_STEPS):
        self.albue.step(-self.HOMING_DIR_ALBUE)
        time.sleep_ms(2)

    while self.sensor_albue.value() == 1:
        self.albue.step(self.HOMING_DIR_ALBUE)
        time.sleep_ms(3)

    self.albue.reset()
    print(">> Albue Nulstillet!")
    print("HOMING COMPLETE")

# After homing, you choose what Cartesian pose corresponds to this physical pose.
# Keep your previous assumption (fully extended along +X):
self.last_x = self.L1 + self.L2
self.last_y = 0.0

```

```

# Initialize lastShoulderSteps/lastElbowSteps from that pose (so first move delta is correct)
ik0 = self._InverseKinematics(self.last_x, self.last_y)
if ik0 is None:
    self.lastShoulderSteps = 0
    self.lastElbowSteps = 0
else:
    sh, el = ik0
    self.lastShoulderSteps, self.lastElbowSteps = self._moveSteppersCount(sh, el)

self._gcode_reset_modal()

# =====
# --- PUBLIC DRAWING API (now uses your IK + step deltas)
# =====
# @block {
#   "label": "sand robot # {id} move line to x {target_x} y {target_y} segments {segments} speed {speed}",
#   "params": {
#     "target_x": {"checkType": "Number"},
#     "target_y": {"checkType": "Number"},
#     "segments": {"checkType": "Number"},
#     "speed": {"checkType": "Number"}
#   }
# }
# Move the robot in a straight line to the target point.
def move_line(self, target_x, target_y, segments=500, speed=None):
    if speed is None:
        speed = self.default_speed_ms

    start_x = self.last_x
    start_y = self.last_y

    deltaX = (target_x - start_x) / segments
    deltaY = (target_y - start_y) / segments

    reached_x = start_x
    reached_y = start_y

    for i in range(1, segments + 1):
        px = start_x + deltaX * i
        py = start_y + deltaY * i

        ik = self._InverseKinematics(px, py)
        if ik is None:
            print("Fejl: Punkt uden for rækkevidde!", px, py)
            break
        theta1, theta2 = ik

        if self.prev_theta1 is None:
            self.prev_theta1, self.prev_theta2 = theta1, theta2
        else:
            theta1 = self._unwrap(theta1, self.prev_theta1)
            theta2 = self._unwrap(theta2, self.prev_theta2) # optional, usually less critical
            self.prev_theta1, self.prev_theta2 = theta1, theta2

        shoulder, elbow = theta1, theta2
        shoulderSteps, elbowSteps = self._moveSteppersCount(shoulder, (elbow - shoulder))

        totalShoulderSteps = shoulderSteps - self.lastShoulderSteps
        totalElbowSteps = elbowSteps - self.lastElbowSteps

        # Your mapping exactly:
        # skulder motor gets totalShoulderSteps
        # albue motor gets (totalShoulderSteps - totalElbowSteps)
        self.skulder.step(totalShoulderSteps)
        self.albue.step(totalElbowSteps)
        #self._step_n(self.skulder, totalShoulderSteps, speed_ms=int(speed))
        #self._step_n(self.albue, totalShoulderSteps - totalElbowSteps, speed_ms=int(speed))

    self.lastShoulderSteps = shoulderSteps
    self.lastElbowSteps = elbowSteps

```

```

        reached_x = px
        reached_y = py

    self.last_x = float(reached_x)
    self.last_y = float(reached_y)

# @block {
#   "label": "sand robot # {id} move arc center x {center_x} center y {center_y} radius {radius} start angle
{start_angle} end angle {end_angle} segments {segments} speed {speed}",
#   "params": {
#     "center_x": {"checkType": "Number"},
#     "center_y": {"checkType": "Number"},
#     "radius": {"checkType": "Number"},
#     "start_angle": {"checkType": "Number"},
#     "end_angle": {"checkType": "Number"},
#     "segments": {"checkType": "Number"},
#     "speed": {"checkType": "Number"}
#   }
# }
# Draw an arc on the sand table.
def move_arc(self, center_x, center_y, radius, start_angle, end_angle, segments=50, speed=None):
    if speed is None:
        speed = self.default_speed_ms

    start_rad = math.radians(start_angle)
    end_rad = math.radians(end_angle)

    start_point_x = center_x + radius * math.cos(start_rad)
    start_point_y = center_y + radius * math.sin(start_rad)
    self.move_line(start_point_x, start_point_y, segments=200, speed=speed)

    for i in range(1, segments + 1):
        fraction = i / segments
        current_angle = start_rad + (end_rad - start_rad) * fraction
        target_x = center_x + radius * math.cos(current_angle)
        target_y = center_y + radius * math.sin(current_angle)
        self.move_line(target_x, target_y, segments=1, speed=speed)

# @block {
#   "label": "sand robot # {id} draw spiral max radius {max_radius} turns {vindinger} segments per turn
{segments_pr_omgang} speed {speed}",
#   "params": {
#     "max_radius": {"checkType": "Number"},
#     "vindinger": {"checkType": "Number"},
#     "segments_pr_omgang": {"checkType": "Number"},
#     "speed": {"checkType": "Number"}
#   }
# }
# Draw a spiral pattern.
def draw_spiral(self, max_radius, vindinger=10, segments_pr_omgang=60, speed=None):
    if speed is None:
        speed = self.default_speed_ms

    total_angle = vindinger * 360
    total_segments = vindinger * segments_pr_omgang

    self.move_line(0, 0, segments=500, speed=speed)

    for i in range(1, total_segments + 1):
        fraction = i / total_segments
        angle_rad = math.radians(total_angle * fraction)
        current_radius = max_radius * fraction
        target_x = current_radius * math.cos(angle_rad)
        target_y = current_radius * math.sin(angle_rad)
        self.move_line(target_x, target_y, segments=1, speed=speed)

# @block {
#   "label": "sand robot # {id} draw flower radius {max_radius} petals {petals} speed {speed}",
#   "params": {
#     "max_radius": {"checkType": "Number"},
#     "petals": {"checkType": "Number"},
#     "speed": {"checkType": "Number"}

```

```

# }
# }
# Draw a flower pattern.
def draw_flower(self, max_radius, petals=5, speed=None):
    if speed is None:
        speed = self.default_speed_ms

    self.move_line(0, 0, segments=500, speed=speed)

    steps = 360
    for i in range(steps + 1):
        theta = math.radians(i)
        r = max_radius * math.cos(petals * theta)
        target_x = r * math.cos(theta)
        target_y = r * math.sin(theta)
        self.move_line(target_x, target_y, segments=1, speed=speed)

# =====
# --- PUBLIC: Run G-code (unchanged)
# =====
# @block {
#     "label": "sand robot # {id} run gcode text {gcode_text} segments {segments} draw speed {draw_speed_ms} travel
speed {travel_speed_ms} reset modal {reset_modal}",
#     "params": {
#         "segments": {"checkType": "Number"},
#         "draw_speed_ms": {"checkType": "Number"},
#         "travel_speed_ms": {"checkType": "Number"},
#         "reset_modal": {"checkType": "Boolean"}
#     }
# }
# Run G-code from a text string.
def run_gcode_text(self, gcode_text, segments=40, draw_speed_ms=None, travel_speed_ms=None, reset_modal=True):
    if reset_modal:
        self._gcode_reset_modal()
    for raw in gcode_text.split('\n'):
        self._gcode_process_line(raw, segments=segments, draw_speed_ms=draw_speed_ms, travel_speed_ms=travel_speed_ms)

# @block {
#     "label": "sand robot # {id} run gcode file {path} segments {segments} draw speed {draw_speed_ms} travel speed
{travel_speed_ms} reset modal {reset_modal}",
#     "params": {
#         "segments": {"checkType": "Number"},
#         "draw_speed_ms": {"checkType": "Number"},
#         "travel_speed_ms": {"checkType": "Number"},
#         "reset_modal": {"checkType": "Boolean"}
#     }
# }
# Run G-code from a file path.
def run_gcode_file(self, path, segments=40, draw_speed_ms=None, travel_speed_ms=None, reset_modal=True):
    if reset_modal:
        self._gcode_reset_modal()
    with open(path, "r") as f:
        for raw in f:
            self._gcode_process_line(raw, segments=segments, draw_speed_ms=draw_speed_ms,
travel_speed_ms=travel_speed_ms)

```

static/page/blocks/libraries/ds1302.py

```

"""DS1302 RTC Driver for MicroPython (3-wire interface)"""

from machine import Pin
import time

# @block {
#     "category": "DS1302 DSL",
#     "color": 35,
#     "url": "https://github.com/micropython/micropython",
#     "instanceMode": "multiple",

```

```

# "methodInstanceMode": "key_input"
# }
# DS1302 real-time clock blocks generated from the Python library.
class DS1302:
    REG_SECOND = 0x80
    REG_MINUTE = 0x82
    REG_HOUR = 0x84
    REG_DAY = 0x86
    REG_MONTH = 0x88
    REG_YEAR = 0x8C
    REG_WP = 0x8E

    # @block {
    #     "params": {
    #         "id": {"checkType": "Number", "defaultValue": 1},
    #         "clk_pin": {"checkType": "Number"},
    #         "dat_pin": {"checkType": "Number"},
    #         "rst_pin": {"checkType": "Number"}
    #     }
    # }
    # Create a DS1302 RTC instance.
    def __init__(self, clk_pin, dat_pin, rst_pin, id=1):
        self.id = id
        self.clk = self._ensure_output_pin(clk_pin)
        self.dat = self._ensure_pin(dat_pin)
        self.rst = self._ensure_output_pin(rst_pin)
        self.rst.value(0)
        self.clk.value(0)
        self._write_byte(self.REG_WP, 0x00)
        sec = self._read_byte(self.REG_SECOND | 0x01)
        if sec & 0x80:
            print("Enabling DS1302 oscillator...")
            self._write_byte(self.REG_SECOND, sec & 0x7F)

    def _ensure_pin(self, value):
        return value if isinstance(value, Pin) else Pin(int(value))

    def _ensure_output_pin(self, value):
        return value if isinstance(value, Pin) else Pin(int(value), Pin.OUT)

    def _write_raw(self, byte):
        self.dat.init(Pin.OUT)
        for _ in range(8):
            self.dat.value(byte & 0x01)
            byte >>= 1
            time.sleep_us(1)
            self.clk.value(1)
            time.sleep_us(1)
            self.clk.value(0)

    def _read_raw(self):
        self.dat.init(Pin.IN)
        byte = 0
        for i in range(8):
            if self.dat.value():
                byte |= (1 << i)
            time.sleep_us(1)
            self.clk.value(1)
            time.sleep_us(1)
            self.clk.value(0)
        return byte

    def _write_byte(self, cmd, data):
        self.rst.value(1)
        time.sleep_us(4)
        self._write_raw(cmd)
        self._write_raw(data)
        self.rst.value(0)
        time.sleep_us(4)

    def _read_byte(self, cmd):
        self.rst.value(1)

```

```

        time.sleep_us(4)
        self._write_raw(cmd)
        result = self._read_raw()
        self.rst.value(0)
        time.sleep_us(4)
        return result

def bcd_to_dec(self, bcd):
    return (bcd // 16) * 10 + (bcd % 16)

def dec_to_bcd(self, dec):
    return (dec // 10) * 16 + (dec % 10)

# @block {
#   "kind": "value"
# }
# Read the current time tuple from the RTC.
def get_time(self):
    second = self.bcd_to_dec(self._read_byte(self.REG_SECOND | 0x01) & 0x7F)
    minute = self.bcd_to_dec(self._read_byte(self.REG_MINUTE | 0x01))
    hour = self.bcd_to_dec(self._read_byte(self.REG_HOUR | 0x01) & 0x3F)
    day = self.bcd_to_dec(self._read_byte(self.REG_DAY | 0x01))
    month = self.bcd_to_dec(self._read_byte(self.REG_MONTH | 0x01))
    year = self.bcd_to_dec(self._read_byte(self.REG_YEAR | 0x01)) + 2000
    return (year, month, day, hour, minute, second)

# @block {
#   "kind": "value"
# }
# Read the current year from the RTC.
def get_year(self):
    return self.get_time()[0]

# @block {
#   "kind": "value"
# }
# Read the current month from the RTC.
def get_month(self):
    return self.get_time()[1]

# @block {
#   "kind": "value"
# }
# Read the current day from the RTC.
def get_day(self):
    return self.get_time()[2]

# @block {
#   "kind": "value"
# }
# Read the current hour from the RTC.
def get_hour(self):
    return self.get_time()[3]

# @block {
#   "kind": "value"
# }
# Read the current minute from the RTC.
def get_minute(self):
    return self.get_time()[4]

# @block {
#   "kind": "value"
# }
# Read the current second from the RTC.
def get_second(self):
    return self.get_time()[5]

# @block {
#   "kind": "value",
#   "params": {
#     "show_seconds": {"checkType": "Boolean", "defaultValue": true}

```

```

# }
# }
# Format the current time as text.
def get_time_text(self, show_seconds=True):
    _, _, hour, minute, second = self.get_time()
    if show_seconds:
        return "%02d:%02d:%02d" % (hour, minute, second)
    return "%02d:%02d" % (hour, minute)

# @block {
#   "kind": "value"
# }
# Format the current date as text.
def get_date_text(self):
    year, month, day, _, _, _ = self.get_time()
    return "%02d/%02d/%04d" % (day, month, year)

# @block {
#   "params": {
#     "year": {"checkType": "Number"},
#     "month": {"checkType": "Number"},
#     "day": {"checkType": "Number"},
#     "hour": {"checkType": "Number"},
#     "minute": {"checkType": "Number"},
#     "second": {"checkType": "Number"}
#   }
# }
# Set the RTC date and time.
def set_time(self, year, month, day, hour, minute, second):
    self._write_byte(self.REG_WP, 0x00)
    self._write_byte(self.REG_SECOND, self.dec_to_bcd(second))
    self._write_byte(self.REG_MINUTE, self.dec_to_bcd(minute))
    self._write_byte(self.REG_HOUR, self.dec_to_bcd(hour))
    self._write_byte(self.REG_DAY, self.dec_to_bcd(day))
    self._write_byte(self.REG_MONTH, self.dec_to_bcd(month))
    self._write_byte(self.REG_YEAR, self.dec_to_bcd(year - 2000))
    self._write_byte(self.REG_WP, 0x80)

```

static/page/blocks/libraries/dfplayer.py

```

"""
DFPlayer Mini MP3 Player Driver for MicroPython
Based on PicoDFPlayer by mannbro (MIT License)
https://github.com/mannbro/PicoDFPlayer
"""

from machine import UART, Pin
from time import sleep_ms

# @block {
#   "category": "DFPlayer DSL",
#   "color": 300,
#   "url": "https://github.com/mannbro/PicoDFPlayer",
#   "instanceMode": "multiple",
#   "methodInstanceMode": "key_input"
# }
# DFPlayer Mini blocks generated from the Python library.
class DFPlayer:
    START_BYTE = 0x7E
    VERSION_BYTE = 0xFF
    COMMAND_LENGTH = 0x06
    ACKNOWLEDGE = 0x00 # No feedback
    END_BYTE = 0xEF

    # @block {
    #   "params": {
    #     "id": {"checkType": "Number", "defaultValue": 1},
    #     "uart_id": {"checkType": "Number", "defaultValue": 1},

```

```

#     "tx_pin": {"checkType": "Number", "defaultValue": 4},
#     "rx_pin": {"checkType": "Number", "defaultValue": 5}
# }
# }
# Create a DFPlayer instance.
def __init__(self, uart_id=1, tx_pin=4, rx_pin=5, id=1):
    """Initialiser DFPlayer.

    Args:
        uart_id: UART nummer (0 eller 1)
        tx_pin: GPIO pin til TX (sender til DFPlayer RX)
        rx_pin: GPIO pin til RX (modtager fra DFPlayer TX)
    """
    self.id = id
    self.uart = UART(uart_id, baudrate=9600,
                     tx=Pin(tx_pin), rx=Pin(rx_pin))
    sleep_ms(200) # Vent paa DFPlayer boot

def _send(self, cmd, param1=0, param2=0):
    """Send kommando til DFPlayer."""
    checksum = -(self.VERSION_BYTE + self.COMMAND_LENGTH + cmd +
                 self.ACKNOWLEDGE + param1 + param2)
    high = (checksum >> 8) & 0xFF
    low = checksum & 0xFF

    data = bytes([
        self.START_BYTE,
        self.VERSION_BYTE,
        self.COMMAND_LENGTH,
        cmd,
        self.ACKNOWLEDGE,
        param1,
        param2,
        high,
        low,
        self.END_BYTE
    ])
    self.uart.write(data)
    sleep_ms(50)

# Afspilning
# @block {
#     "params": {
#         "track": {"checkType": "Number", "defaultValue": 1}
#     }
# }
def play(self, track=1):
    """Afspil track nummer (1-3000)."""
    self._send(0x03, (track >> 8) & 0xFF, track & 0xFF)

# @block {
#     "params": {
#         "folder": {"checkType": "Number"},
#         "track": {"checkType": "Number"}
#     }
# }
def play_folder(self, folder, track):
    """Afspil track fra en mappe (mappe 1-99, track 1-255)."""
    self._send(0x0F, folder, track)

# @block {
#     "params": {
#         "track": {"checkType": "Number"},
#         "loop": {"checkType": "Boolean", "defaultValue": false}
#     }
# }
def play_mp3(self, track, loop=False):
    """Afspil fra /mp3 mappen (track 1-9999).

    Args:
        track: Track nummer (1-9999)
        loop: Gentag track indtil stop() kaldes
    """

```



```

"""
self._send(0x12, (track >> 8) & 0xFF, track & 0xFF)
if loop:
    sleep_ms(100) # Vent paa at afspilning starter
    self._send(0x19, 0, 0) # Aktiver single loop

# @block {}
def pause(self):
    """Pause afspilning."""
    self._send(0x0E)

# @block {}
def resume(self):
    """Fortsat afspilning."""
    self._send(0x0D)

# @block {}
def stop(self):
    """Stop afspilning."""
    self._send(0x16)

# @block {
#   "params": {
#     "enable": {"checkType": "Boolean", "defaultValue": true}
#   }
# }
def loop_current(self, enable):
    """Aktiver eller deaktivert loop af aktuel track (True/False).
    Skal kaldes EFTER play/play_mp3."""
    self._send(0x19, 0, 0 if enable else 1) # 0 = loop on, 1 = loop off

# @block {}
def next(self):
    """Naeste track."""
    self._send(0x01)

# @block {}
def prev(self):
    """Forrige track."""
    self._send(0x02)

# Lydstyrke
# @block {
#   "params": {
#     "level": {"checkType": "Number"}
#   }
# }
def volume(self, level):
    """Saet lydstyrke (0-30)."""
    if level < 0:
        level = 0
    if level > 30:
        level = 30
    self._send(0x06, 0, level)

# @block {}
def volume_up(self):
    """Skru op."""
    self._send(0x04)

# @block {}
def volume_down(self):
    """Skru ned."""
    self._send(0x05)

# Equalizer
# @block {
#   "params": {
#     "mode": {
#       "inputKind": "field_dropdown",
#       "options": [{"Normal", "0"}, {"Pop", "1"}, {"Rock", "2"}, {"Jazz", "3"}, {"Classic", "4"}, {"Bass", "5"}],
#       "checkType": "Number",

```

```

#         "defaultValue": 0
#     }
# }
# }
def eq(self, mode):
    """Saet equalizer (0=Normal, 1=Pop, 2=Rock, 3=Jazz, 4=Classic, 5=Bass)."""
    self._send(0x07, 0, mode)

# System
# @block {}
def reset(self):
    """Reset DFPlayer."""
    self._send(0x0C)
    sleep_ms(1500) # Vent paa reset

# @block {}
def sleep(self):
    """Gaa i standby."""
    self._send(0x0A)

# @block {}
def wake(self):
    """Vaagn fra standby."""
    self._send(0x0B)

```

static/page/blocks/libraries/st7735s.py

```

"""ST7735S TFT Display Driver for MicroPython (128x160, SPI, 16-bit color)"""

import time
from machine import Pin

# @block {
#     "category": "ST7735S DSL",
#     "color": 210,
#     "url": "https://github.com/micropython/micropython",
#     "instanceMode": "multiple",
#     "methodInstanceMode": "key_input"
# }
# ST7735S display blocks generated from the Python library.
class ST7735S:
    # @block {
    #     "params": {
    #         "id": {"checkType": "Number", "defaultValue": 1},
    #         "spi": {"inputKind": "variable"},
    #         "dc_pin": {"checkType": "Number"},
    #         "rst_pin": {"checkType": "Number"},
    #         "cs_pin": {"checkType": "Number"},
    #         "width": {"checkType": "Number", "defaultValue": 160},
    #         "height": {"checkType": "Number", "defaultValue": 128}
    #     }
    # }
    # Create an ST7735S display instance.
    def __init__(self, id, spi, dc_pin, rst_pin, cs_pin, width=160, height=128):
        self.id = id
        self.spi = spi
        self.dc = self._ensure_output_pin(dc_pin)
        self.rst = self._ensure_output_pin(rst_pin)
        self.cs = self._ensure_output_pin(cs_pin)
        self.width = width
        self.height = height
        self.line_buffer = bytearray(self.width * 2)
        self.init_display()

    def _ensure_output_pin(self, value):
        return value if isinstance(value, Pin) else Pin(int(value), Pin.OUT)

    def write_cmd(self, cmd):

```

```

        self.cs.value(0)
        self.dc.value(0)
        self.spi.write(bytearray([cmd]))
        self.cs.value(1)

def write_data(self, data):
    self.cs.value(0)
    self.dc.value(1)
    self.spi.write(data)
    self.cs.value(1)

def init_display(self):
    self.rst.value(1)
    time.sleep(0.01)
    self.rst.value(0)
    time.sleep(0.01)
    self.rst.value(1)
    time.sleep(0.12)

    self.write_cmd(0x01) # Software reset
    time.sleep(0.15)

    self.write_cmd(0x11) # Sleep out
    time.sleep(0.05)

    self.write_cmd(0x3A) # Color mode
    self.write_data(bytearray([0x05])) # 16-bit color

    self.write_cmd(0x36) # Memory access control (rotation)
    self.write_data(bytearray([0x60])) # Landscape mode

    self.write_cmd(0x20) # Inversion off
    self.write_cmd(0x13) # Normal display mode

    self.write_cmd(0x29) # Display on
    time.sleep(0.05)

    print("ST7735S display initialized (160x128)")

def set_window(self, x0, y0, x1, y1):
    self.write_cmd(0x2A)
    self.write_data(bytearray([0x00, x0, 0x00, x1]))
    self.write_cmd(0x2B)
    self.write_data(bytearray([0x00, y0, 0x00, y1]))
    self.write_cmd(0x2C)

# @block {
#     "params": {
#         "color": {"checkType": "Number"}
#     }
# }
def fill(self, color):
    self.set_window(0, 0, self.width - 1, self.height - 1)
    for i in range(0, self.width * 2, 2):
        self.line_buffer[i] = color >> 8
        self.line_buffer[i + 1] = color & 0xFF
    self.cs.value(0)
    self.dc.value(1)
    for _ in range(self.height):
        self.spi.write(self.line_buffer)
    self.cs.value(1)

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "w": {"checkType": "Number"},
#         "h": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }
def fill_rect(self, x, y, w, h, color):

```

```

        if x < 0 or y < 0 or x + w > self.width or y + h > self.height:
            return
        self.set_window(x, y, x + w - 1, y + h - 1)
        line_buf = bytearray(w * 2)
        for i in range(0, w * 2, 2):
            line_buf[i] = color >> 8
            line_buf[i + 1] = color & 0xFF
        self.cs.value(0)
        self.dc.value(1)
        for _ in range(h):
            self.spi.write(line_buf)
        self.cs.value(1)

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }
def pixel(self, x, y, color):
    if x < 0 or y < 0 or x >= self.width or y >= self.height:
        return
    self.set_window(x, y, x, y)
    self.write_data(bytearray([color >> 8, color & 0xFF]))

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "w": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }
def hline(self, x, y, w, color):
    self.fill_rect(x, y, w, 1, color)

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "h": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }
def vline(self, x, y, h, color):
    self.fill_rect(x, y, 1, h, color)

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "w": {"checkType": "Number"},
#         "h": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }
def rect(self, x, y, w, h, color):
    self.hline(x, y, w, color)
    self.hline(x, y + h - 1, w, color)
    self.vline(x, y, h, color)
    self.vline(x + w - 1, y, h, color)

# @block {
#     "params": {
#         "x0": {"checkType": "Number"},
#         "y0": {"checkType": "Number"},
#         "x1": {"checkType": "Number"},
#         "y1": {"checkType": "Number"},
#         "color": {"checkType": "Number"}
#     }
# }

```

```

def line(self, x0, y0, x1, y1, color):
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy
    while True:
        self.pixel(x0, y0, color)
        if x0 == x1 and y0 == y1:
            break
        e2 = 2 * err
        if e2 > -dy:
            err -= dy
            x0 += sx
        if e2 < dx:
            err += dx
            y0 += sy

# @block {
#     "params": {
#         "x": {"checkType": "Number"},
#         "y": {"checkType": "Number"},
#         "color": {"checkType": "Number"},
#         "size": {"checkType": "Number", "defaultValue": 1}
#     }
# }
def text(self, string, x, y, color, size=1):
    for i, char in enumerate(string):
        self.draw_char(char, x + i * 8 * size, y, color, size)

def draw_char(self, c, x, y, color, size=1):
    font = self.get_font_data(ord(c))
    for col in range(8):
        for row in range(8):
            if font[row] & (1 << (7 - col)):
                if size == 1:
                    self.pixel(x + col, y + row, color)
                else:
                    self.fill_rect(x + col * size, y + row * size, size, size, color)

def get_font_data(self, char_code):
    if 48 <= char_code <= 57:
        fonts = {
            48: [0x3C, 0x66, 0x6E, 0x76, 0x66, 0x66, 0x3C, 0x00],
            49: [0x18, 0x38, 0x18, 0x18, 0x18, 0x18, 0x7E, 0x00],
            50: [0x3C, 0x66, 0x06, 0x0C, 0x18, 0x30, 0x7E, 0x00],
            51: [0x3C, 0x66, 0x06, 0x1C, 0x06, 0x66, 0x3C, 0x00],
            52: [0x0C, 0x1C, 0x2C, 0x4C, 0x7E, 0x0C, 0x0C, 0x00],
            53: [0x7E, 0x60, 0x7C, 0x06, 0x06, 0x66, 0x3C, 0x00],
            54: [0x3C, 0x60, 0x60, 0x7C, 0x66, 0x66, 0x3C, 0x00],
            55: [0x7E, 0x06, 0x0C, 0x18, 0x30, 0x30, 0x30, 0x00],
            56: [0x3C, 0x66, 0x66, 0x3C, 0x66, 0x66, 0x3C, 0x00],
            57: [0x3C, 0x66, 0x66, 0x3E, 0x06, 0x06, 0x3C, 0x00],
        }
        return fonts[char_code]
    elif 65 <= char_code <= 90:
        fonts = {
            65: [0x18, 0x3C, 0x66, 0x66, 0x7E, 0x66, 0x66, 0x00],
            66: [0x7C, 0x66, 0x66, 0x7C, 0x66, 0x66, 0x7C, 0x00],
            67: [0x3C, 0x66, 0x60, 0x60, 0x60, 0x66, 0x3C, 0x00],
            68: [0x78, 0x6C, 0x66, 0x66, 0x66, 0x6C, 0x78, 0x00],
            69: [0x7E, 0x60, 0x60, 0x7C, 0x60, 0x60, 0x7E, 0x00],
            70: [0x7E, 0x60, 0x60, 0x7C, 0x60, 0x60, 0x60, 0x00],
            71: [0x3C, 0x66, 0x60, 0x6E, 0x66, 0x66, 0x3C, 0x00],
            72: [0x66, 0x66, 0x66, 0x7E, 0x66, 0x66, 0x66, 0x00],
            73: [0x3C, 0x18, 0x18, 0x18, 0x18, 0x18, 0x3C, 0x00],
            74: [0x1E, 0x0C, 0x0C, 0x0C, 0x0C, 0x6C, 0x38, 0x00],
            75: [0x66, 0x6C, 0x78, 0x70, 0x78, 0x6C, 0x66, 0x00],
            76: [0x60, 0x60, 0x60, 0x60, 0x60, 0x60, 0x7E, 0x00],
            77: [0x63, 0x77, 0x7F, 0x6B, 0x63, 0x63, 0x63, 0x00],
            78: [0x66, 0x76, 0x7E, 0x7E, 0x6E, 0x66, 0x66, 0x00],
            79: [0x3C, 0x66, 0x66, 0x66, 0x66, 0x66, 0x3C, 0x00],

```

```

80: [0x7C, 0x66, 0x66, 0x7C, 0x60, 0x60, 0x60, 0x00],
81: [0x3C, 0x66, 0x66, 0x66, 0x66, 0x3C, 0x0E, 0x00],
82: [0x7C, 0x66, 0x66, 0x7C, 0x6C, 0x66, 0x63, 0x00],
83: [0x3C, 0x66, 0x60, 0x3C, 0x06, 0x66, 0x3C, 0x00],
84: [0x7E, 0x18, 0x18, 0x18, 0x18, 0x18, 0x18, 0x00],
85: [0x66, 0x66, 0x66, 0x66, 0x66, 0x66, 0x3C, 0x00],
86: [0x66, 0x66, 0x66, 0x66, 0x66, 0x3C, 0x18, 0x00],
87: [0x63, 0x63, 0x63, 0x6B, 0x7F, 0x77, 0x63, 0x00],
88: [0x66, 0x66, 0x3C, 0x18, 0x3C, 0x66, 0x66, 0x00],
89: [0x66, 0x66, 0x66, 0x3C, 0x18, 0x18, 0x18, 0x00],
90: [0x7E, 0x06, 0x0C, 0x18, 0x30, 0x60, 0x7E, 0x00],
}
return fonts.get(char_code, [0xFF] * 8)
else:
    fonts = {
        32: [0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00],
        33: [0x18, 0x18, 0x18, 0x18, 0x00, 0x18, 0x18, 0x00],
        43: [0x00, 0x18, 0x18, 0x7E, 0x18, 0x18, 0x00, 0x00],
        45: [0x00, 0x00, 0x00, 0x7E, 0x00, 0x00, 0x00, 0x00],
        46: [0x00, 0x00, 0x00, 0x00, 0x00, 0x18, 0x18, 0x00],
        47: [0x00, 0x06, 0x0C, 0x18, 0x30, 0x60, 0x00, 0x00],
        58: [0x00, 0x18, 0x18, 0x00, 0x18, 0x18, 0x00, 0x00],
        63: [0x3C, 0x66, 0x06, 0x0C, 0x18, 0x00, 0x18, 0x00],
        94: [0x18, 0x3C, 0x66, 0x00, 0x00, 0x00, 0x00, 0x00],
    }
    return fonts.get(char_code, [0xFF] * 8)

```

```

# @block {
#   "category": "ST7735S DSL",
#   "color": 210,
#   "kind": "value",
#   "params": {
#     "r": {"checkType": "Number"},
#     "g": {"checkType": "Number"},
#     "b": {"checkType": "Number"}
#   }
# }

# Convert RGB888 values into a 16-bit display color.
def color565(r, g, b):
    """Convert RGB888 to RGB565 color format."""
    return ((r & 0xF8) << 8) | ((g & 0xFC) << 3) | (b >> 3)

# Color constants
BLACK = color565(0, 0, 0)
WHITE = color565(255, 255, 255)
RED = color565(255, 0, 0)
GREEN = color565(0, 255, 0)
BLUE = color565(0, 0, 255)
YELLOW = color565(255, 255, 0)
CYAN = color565(0, 255, 255)
MAGENTA = color565(255, 0, 255)
ORANGE = color565(255, 165, 0)
GRAY = color565(128, 128, 128)

```